CT Image Super-Resolution via Computational Imaging Methods: Variational, End-to-End and Generative Approaches Compared

Project Report — Computational Imaging 2025/2026

Assignment: Group H Project Template

Group	H
Students	Pietro Cimino, Mattia Boffa
Professors	Prof. Elena Loli Piccolomini, Prof. Davide Evangelista
Department	Department of Computer Science and Engineering (DISI)
Academic Year	2025 – 2026
Dataset	Mayo Low-Dose CT Dataset
Task	Super-resolution (inverse problem)

1. Introduction and Project Objectives

This work addresses a **super-resolution** (SR) problem on low-dose computed tomography (CT) images, as part of the exam project for the Computational Imaging course (A.Y. 2025/2026), following the assignment given to **Group H**. The project, carried out by a group of two students (Pietro Cimino and Mattia Boffa), requires a systematic comparison of several methodological families — classical and modern — for reconstructing high-resolution images from degraded observations, with the goal not only of achieving good reconstruction performance, but also of critically reasoning about the strengths and limitations of each approach.

As required by the assignment for a two-student group, **three** of the four planned methods were implemented and compared (excluding the hybrid Plug-and-Play in Half Quadratic Splitting method), representative of the three major families of computational imaging:

- **Variational method**: Total p-Variation (TpV) regularization, solved with the Chambolle-Pock algorithm;
- **End-to-End method**: an *Attention U-Net* neural network trained in a supervised fashion;
- **Generative method**: a Generative Adversarial Network (GAN) combined with Diffusion Posterior Sampling (DPS), adapted to the inverse problem via latent-space optimization.

All methods were evaluated on the same degraded observations, for the same downscaling factor and noise level, to ensure a fair comparison, and with the same quantitative metrics (PSNR and SSIM), in addition to a qualitative visual comparison.

2. Description of the Assigned Track (Group H)

The assignment document (*Group H Project Template*) precisely defines the elements the project must satisfy. Below is a summary of the requirements, individually checked against the work carried out.

2.1 General Requirements

- **Number of students**: 2 (Pietro Cimino, Mattia Boffa) — consistent with the required group composition;
- **Dataset**: Mayo (low-dose CT) — used as specified in the assignment;
- **Task**: super-resolution, formulated as the inverse problem $y = K(x) + e$, with x the high-resolution image (ground truth), K the downscaling operator, e additive Gaussian noise, and y the degraded low-resolution observation;
- **Image size**: all images were resized to 256×256, as required;
- **Degradation parameters**: SR factor tested for values 2 and 4; noise level tested for values 0.005 and 0.01 — both as specified in the assignment;
- **Consistency of the degraded observations**: all methods were evaluated on the same degraded observations (same operator K and same added noise), ensuring a fair comparison between methods.

2.2 Methodological Requirements

The assignment requires, for a two-student group, the choice of three out of the four proposed methods (mandatorily excluding one among End-to-End, Generative, and Hybrid). The group chose to implement the **Variational** method (mandatory), the **End-to-End** method, and the **Generative** method, excluding the **Hybrid (Plug-and-Play in HQS)** method.

Method Required by the Assignment	Status in the Project	Implementation
Variational — TpV, p {0.1, 0.4}	Included	Chambolle-Pock, file TpV.py
End-to-End — UNet / ViT / NAF-Net	Included (UNet)	Attention U-Net, file EtE.py
Generative — GAN with DPS	Included	GAN (StyleGAN-like) + latent DPS, file GAN.py
Hybrid — Plug-and-Play in HQS	Excluded (optional for groups of 2)	—

2.3 Required Deliverables

- Quantitative comparison (PSNR, SSIM) between methods — presented in Chapter 8;
- Visual comparison (reconstructed images) — presented in Chapter 9;
- Overall comparison plot across all methods — presented in Chapters 9 and 10;
- Report with methods, experimental setup, and discussion — the present document;
- Documented code (ideally on GitHub) — files `EtE.py`, `GAN.py`, `MyUtilities.py`, `TpV.py`, described in Chapter 6;
- PowerPoint presentation discussing the results — produced separately (*Presentazione.pptx*).

3. Dataset and Pre-Processing Pipeline

The dataset used is the **Mayo Low-Dose CT Dataset**, composed of grayscale computed tomography slices. The on-disk structure includes per-patient/series subfolders, explored via `glob` patterns of the form `data_path/*/*.png`.

3.1 MayoDataset Class (End-to-End)

In the file `MyUtilities.py`, the class `MayoDataset` implements the entire degradation pipeline used to train the End-to-End network:

- Loading the grayscale image (`convert('L')`) and resizing it to 256×256 (high resolution, HR);
- Applying the `operators.DownScaling` operator (from `IPPy`) to obtain the low-resolution (LR) observation, according to the requested downscaling factor (2 or 4);
- Adding additive Gaussian noise via `utilities.gaussian_noise`, with a standard deviation equal to the configured noise level (0.005 or 0.01);
- Upsampling the LR image back to 256×256 (via `transforms.Resize`), so that the UNet receives an input tensor of the same size as the target image, according to the residual-learning scheme (see Chapter 5.2).

The same degradation scheme (downscaling + additive Gaussian noise) is shared, with the appropriate implementation variants, by the files `GAN.py` and `TpV.py` as well, ensuring that all methods operate on exactly the same degraded observations for a given scale and noise level, as required by the assignment.

3.2 Train / Validation / Test Split

For the End-to-End method, the training dataset is split into a training set and a validation set with an 85% / 15% ratio via scikit-learn's `train_test_split` (`random_state=42` for reproducibility), while the test set is loaded from a separate folder (`./Mayo/test`), ensuring that the test images are never seen during training or validation. For the generative method (GAN) and the variational method (TpV), evaluation is performed directly on the same test set, so that the comparison between methods remains consistent.

4. Inverse Problem Formulation

The super-resolution problem is formulated, in line with the assignment, as a linear inverse problem with additive noise:

$$\mathbf{y} = \mathbf{K}(\mathbf{x}) + \mathbf{e}$$

- **x**: high-resolution ground-truth image (256×256);
- **K**: linear downscaling operator (implemented in `IPPy.operators.DownScaling`), parametrized by the scale factor (2 or 4);
- **e**: additive Gaussian noise, with standard deviation {0.005, 0.01};
- **y**: low-resolution, degraded and noisy observation.

The goal of each method is to estimate a reconstruction x of x from the observation y alone (and possibly from knowledge of the operator K), following different strategies: explicit regularization (TpV), direct supervised learning of the inverse map (UNet), or inversion via a pre-trained generative model (GAN + DPS).

5. Implemented Methods

5.1 Variational Method: Total p-Variation (TpV)

The variational method is implemented in the file `TpV.py` and is based on minimizing a functional composed of a data-fidelity term and a Total p-Variation regularization term, with exponent p < 1 (a non-convex regularization, more aggressive than classical Total Variation with p = 1 in preserving edges while reducing noise at the same time):

- **Algorithm**: unconstrained Chambolle-Pock (`solvers.ChambollePockTpVUnconstrained`, from `IPPy`), 150 iterations per configuration;
- **Explored parameters**: exponent p {0.1, 0.4}; regularization parameter , with a dedicated grid search ({1e-3, 8e-2, 9e-2} for scale 4; {5e-3, 9e-2, 4e-1} for scale 2);
- **No training required**: the method is purely model-based, requires no learning data, and is applied directly to the degraded observation y for every combination (scale, noise, , p).

The file implements an `eval(...)` function that, for every combination of noise, and p, generates the degraded observation y_ , runs the solver, and computes PSNR/SSIM against the ground truth, accumulating the results in a nested dictionary `GlobalResults`, which is then serialized to JSON. At the end, for each noise level the values are ranked by the average (PSNR + SSIM)/2, thus identifying the best for each configuration.

5.2 End-to-End Method: Attention U-Net

Among the architectures proposed by the assignment (UNet, ViT, NAF-Net), an **Attention-Gated U-Net** was chosen — an architecture that combines the proven effectiveness of the classical U-Net for image-to-image regression tasks with an attention mechanism on the skip connections, particularly useful for denoising and super-resolution of medical images, where it is important to preserve fine anatomical detail while suppressing the noise introduced by the degradation.

Architecture (file EtE.py)

- **DoubleConv**: base block composed of two consecutive 3×3 convolutions with an intermediate ReLU activation;
- **Encoder**: an input `DoubleConv` (1 64 channels) followed by two `downsample` blocks (2×2 MaxPool2d + DoubleConv), taking the channel count from 64 128 256;
- **Bottleneck**: an additional `downsample` block that raises the channel count to 512, at the minimum spatial resolution;
- **Decoder with Attention Gate**: instead of the classical encoder-decoder concatenation, each decoder level uses an `AttentionGate` block that (i) applies PixelShuffle for upsampling, (ii) filters both the upsampled signal and the skip signal with 1×1 convolutions, (iii) combines the two maps with a ReLU followed by a 1×1 convolution and a sigmoid, generating an attention map between 0 and 1, (iv) multiplies this map by the original skip features, letting the network learn to focus on the most informative regions before the final fusion via `DoubleConv`;
- **Residual output**: the network does not directly predict the reconstructed image, but estimates the noise present in the input observation; the final image is obtained by subtraction, $x = y \quad f(y)$, a typical residual-learning scheme that simplifies the network's task by focusing it on the degradation component rather than on the entire image content.

Loss Function

The loss function, defined in the `Losses` class of `MyUtilities.py`, combines three weighted components:

- **MSE** (weight 0.8): pixel-wise mean squared error, for global fidelity to the reconstruction;
- **Fourier Loss** (weight 1.0): mean difference between the magnitudes of the 2D Fourier transforms of prediction and target, to penalize the loss of high-frequency detail (fine texture);
- **SSIM Loss** (weight 1.0): 1 local SSIM, computed via moving averages and variances (average pooling with a 7×7 window), to align the reconstruction with human structural perception.

Training Setup

Hyperparameter	Value
Optimizer	Adam, lr = 5e-4
Scheduler	CosineAnnealingLR (T_max = 4)
Epochs	8
Batch size	32
Loss	0.8·MSE + 1·FourierLoss + 1·SSIMLoss
Model selection	Best model saved based on validation loss
Trained models	4 independent checkpoints (one per scale/noise combination)

The `MyTrainer` class (in `MyUtilities.py`) manages the entire training and validation loop, logging of metrics and losses to JSON files, saving of loss-curve plots, and, via the `testOnImage` method, qualitative evaluation on a single reference image (`0.png`).

5.3 Generative Method: GAN + Diffusion Posterior Sampling (DPS)

The generative method, implemented in `GAN.py`, is organized into two distinct phases: (1) unconditional training of a generative GAN on high-resolution CT images; (2) use of the pre-trained generator, with frozen weights, as a generative *prior* to solve the super-resolution inverse problem through a guided sampling procedure inspired by Diffusion Posterior Sampling.

GAN Architecture

- **Generator** (`StableGenerator`): starting from a latent vector z (dimension 128), a linear projection produces a 512×4×4 map, followed by 6 residual blocks `GResidualBlock` with GroupNorm, SiLU activation, and 2× bilinear upsampling, up to the 256×256 single-channel resolution, with a final sigmoid activation;
- **Discriminator/Critic** (`StableCritic`): a convolutional stem followed by 6 residual blocks `DResidualBlock` with spectral normalization and AvgPool2d, progressively reducing the resolution, ending with a linear head;
- **Adversarial loss**: hinge loss (WGAN-like) with an R1 gradient penalty on the discriminator with respect to real data, applied every 16 steps, to stabilize training;
- **Generator EMA**: a copy `G_ema` of the generator weights is maintained, updated with an exponential moving average (decay 0.999), later used for inference in the reconstruction phase, following standard practice in GAN training to improve the quality and stability of generated samples.

Hyperparameter	Value
Generator optimizer	Adam, lr = 3e-4, =(0.0, 0.99)
Critic optimizer	Adam, lr = 5e-5, =(0.0, 0.99)
Epochs	50
Batch size	32

Hyperparameter	Value
Precision	Mixed precision (AMP) with separate GradScalers for G and Critic
R1 weight	5, applied every 16 steps

Reconstruction via Latent Optimization (DPS)

Since the GAN is trained unconditionally (it does not receive the degraded observation as input), the reconstruction for the inverse problem is obtained by iteratively optimizing the latent vector z of the frozen generator, so that the generated image G(z) is consistent with the observation y through the degradation operator K. The function `gan_dps_reconstruct` implements this procedure:

- Initialization of z N(0, I) with gradient enabled;
- At each step t, computation of the current estimated image x = G(z), clamped to [0, 1];
- Computation of the data-fidelity loss: K(x) y ² / (2 _y²), backpropagated with respect to z through the generator and the operator K;
- Update of z according to a Langevin-type dynamics: z z step_size· z + (2·step_size)·noise, with step_size proportional to _t², following a decreasing geometric schedule of _t from _start to _end (broad initial exploration, refinement in the final iterations);
- At the end of the iterations (1000 steps in the final configuration), the stochastic noise is set to zero and x_final = G(z) is returned as the final reconstruction.

This scheme effectively implements an adaptation of the Diffusion Posterior Sampling framework to the case where the generative model is not a diffusion model but a GAN: the GAN's latent space is explored with a noisy dynamics guided by the gradient of the data-fidelity term, rather than propagating noise directly in image space as in classical diffusion models.

6. Code Organization

The project code is organized into four main Python files, in addition to the support library `IPPy` (Inverse Problems in Python), used for the degradation operators, classical solvers, and evaluation metrics.

File	Main Contents
MyUtilities.py	MayoDataset class (degradation pipeline), MyTrainer class (training/validation/test loop, logging, single-image evaluation), loss functions (FourierLoss, SSIMLoss, Losses class)
EtE.py	Definition of the Attention U-Net architecture (DoubleConv, downsample, upsample, AttentionGate, UNet) and configuration/execution script for training and testing the End-to-End method
GAN.py	Local MayoDataset class, StableGenerator/StableCritic architecture, adversarial training loop with AMP and R1 penalty, gan_dps_reconstruct function for latent-optimization reconstruction, quantitative evaluation and figure saving
TpV.py	Local MayoDataset class, eval function running the Chambolle-Pock TpV solver over multiple noise, , and p configurations, aggregation and ranking of results, JSON saving

Each script follows the same general scheme: (i) definition of the dataset and data loader, (ii) definition or loading of the model, (iii) optional training, (iv) quantitative evaluation (PSNR/SSIM) on the test set and on a single reference image (`0.png`), (v) saving of results, figures, and logs in JSON format, to ensure traceability and reproducibility of the experiments. This common structure facilitates the comparison between methods, since all of them produce homogeneous outputs (metrics, saved images, log files) used later in Chapters 8 and 9.

7. Experimental Setup

The experimental evaluation was carried out on the super-resolved reconstruction of abdominal CT slices from the Mayo dataset (single-channel images, 256×256), according to the following parameter grid, common to all methods:

Parameter	Tested Values
Downscaling factor	2, 4
Noise level	0.005, 0.01
TpV exponent p	0.1, 0.4
Image size	256 × 256

The applied degradation model is, for each combination, the downscaling operator K (factor 2 or 4) followed by the addition of additive Gaussian noise of level . The same degraded observations are used consistently by all three methods, to ensure the comparability of results. The test-set images are kept separate from the training and validation images for the entire duration of the experiment (only the End-to-End method requires training/validation phases; the TpV and GAN+DPS methods do not require task-specific supervised training, although the GAN still requires a preliminary unconditional generative training).

For each configuration, in addition to the average metrics on the first batch of the test set, the pointwise evaluation on a common reference image (`0.png`, shown in Figure 1) is also reported, which is also used for the qualitative visual comparison in Chapter 9.

Figure 1 – Reference image (ground truth, 512×512, resized to 256×256 for the experiments): axial thoracic CT slice used for the pointwise visual comparison between methods.

8. Quantitative Results

8.1 End-to-End Method (Attention U-Net)

Average PSNR and SSIM on the first batch of the test set, and on the reference image `0.png` (Attention U-Net, 8 training epochs).

Scale	Noise	Test set — PSNR / SSIM	0.png — PSNR / SSIM
×2	0.005	33.19 dB / 0.9460	33.57 dB / 0.9487
×2	0.01	39.78 dB / 0.9723	39.90 dB / 0.9734
×4	0.005	33.98 dB / 0.9199	34.25 dB / 0.9212
×4	0.01	33.74 dB / 0.9139	34.00 dB / 0.9154

Key Findings

- PSNR and SSIM remain stable (or improve) when moving from scale ×2 to ×4, indicating that the network effectively compensates for more severe degradation;
- The combination scale ×2, noise 0.01 gives the best result overall (39.78 dB / 0.9723), clearly higher than scale ×2, noise 0.005;
- The metrics on the single image 0.png are consistently in line with the test-set batch average, confirming the model's stability;
- Each (scale, noise) combination corresponds to an independently trained checkpoint (4 variants overall).

8.2 Variational Method (TpV)

Average PSNR and SSIM on the first batch of the test set (Chambolle-Pock algorithm, 150 iterations, best = 9×10² for both scale factors).

Scale	Noise	p = 0.1 — PSNR / SSIM	p = 0.4 — PSNR / SSIM
×2	0.005	31.06 dB / 0.8956	31.32 dB / 0.8932
×2	0.01	30.88 dB / 0.8870	31.19 dB / 0.8882
×4	0.005	26.85 dB / 0.7790	27.13 dB / 0.7845
×4	0.01	26.78 dB / 0.7725	27.06 dB / 0.7785

Key Findings

- = 9×10² is the best regularization value for every scale/noise combination tested (among {1e-3, 8e-2, 9e-2} for scale ×4 and {5e-3, 9e-2, 4e-1} for scale ×2);
- The exponent p = 0.4 consistently outperforms p = 0.1, in both PSNR and SSIM;
- Performance is dominated by the downscaling factor: scale ×2 reaches about 31 dB, while scale ×4 drops to about 27 dB;
- Sensitivity to the noise level (0.005 vs 0.01) is instead relatively small compared to the effect of the scale factor.

8.3 Generative Method (GAN + DPS)

Average PSNR and SSIM on the first batch of the test set (latent optimization, 1000 steps, = 1×10²).

Scale	Noise	Test set — PSNR / SSIM
×2	0.005	19.70 dB / 0.5795
×2	0.01	20.00 dB / 0.5872
×4	0.005	19.72 dB / 0.5769
×4	0.01	20.24 dB / 0.5909

Key Findings

- It is clearly the weakest method among the three: about 20 dB PSNR, versus about 27–31 dB for TpV and about 33–40 dB for the U-Net;
- The reconstructions are over-smoothed: the generative prior recovers the overall anatomy but loses the fine lung texture (visible in the visual comparison in Chapter 9);
- The method is nearly insensitive to the scale factor (×2 vs ×4) and to the noise level (0.005 vs 0.01): the bottleneck is the expressiveness of the frozen generator, not the severity of the degradation;
- The data-fidelity loss K(G(z)) y² / (2 ²) optimizes the latent vector z of the frozen generator, with K equal to the downscaling operator corresponding to each scale.

8.4 Numerical Comparison Across the Three Methods

The table below summarizes, for each configuration (scale, noise), the average performance of all three methods, using the optimal exponent (p = 0.4) for the TpV method.

Configuration	PSNR TpV	PSNR E2E	PSNR GAN+DPS	SSIM TpV	SSIM E2E	SSIM GAN+DPS
Scale 2, Noise 0.005	31.32	33.19	19.70	0.8932	0.9460	0.5795
Scale 2, Noise 0.01	31.19	39.78	20.00	0.8882	0.9723	0.5872
Scale 4, Noise 0.005	27.13	33.98	19.72	0.7845	0.9199	0.5769
Scale 4, Noise 0.01	27.06	33.74	20.24	0.7785	0.9139	0.5909

Key Findings

- The U-Net (End-to-End) delivers the highest PSNR in every tested configuration;
- The SSIM gap mirrors the one observed in PSNR;
- Even without using any training data, the TpV method stays within about 6 dB of the trained U-Net, and clearly ahead of the GAN-based inversion;
- GAN+DPS is nearly insensitive to scale and noise, confirming that its bottleneck is the expressiveness of the generator, not the severity of the degradation.

9. Visual Results and Qualitative Comparison

The qualitative comparison was carried out on the reference image `0.png` (Figure 1), for each combination of scale and noise, comparing each method's reconstruction with the ground truth and, for the TpV method, also with the original degraded observation.

9.1 Attention U-Net

The reconstructions produced by the U-Net are visually very close to the ground truth in all tested configurations, with good preservation of lung tissue detail even under the most severe degradation conditions (scale ×4), consistent with the quantitative values reported in Chapter 8.1.

9.2 Total p-Variation

The TpV reconstructions show a good trade-off between noise reduction and preservation of the main anatomical edges, but with a more marked loss of fine texture detail compared to the U-Net, especially for the ×4 scale factor, where the starting observation is heavily undersampled. The exponent p = 0.4 produces slightly sharper reconstructions than p = 0.1, confirming what was observed in the quantitative metrics.

9.3 GAN + DPS

The reconstructions obtained through latent-space optimization show plausible anatomy but are excessively smoothed (over-smoothing), with a noticeable loss of fine lung texture and some secondary structural details. The training curves (generator and critic losses over 50 epochs) show a stable adversarial dynamics: the critic loss starts near 2.0 and drops sharply around epochs 10–13, while the generator loss rises from negative values and stabilizes around 1.0–1.2 after about epoch 15, with no signs of training collapse.

9.4 Overall Visual Comparison

For each of the four configurations (scale × noise), a side-by-side comparison was produced between ground truth, TpV reconstruction, U-Net reconstruction, and GAN+DPS reconstruction on the same test image, visually confirming the quality hierarchy observed quantitatively: U-Net > TpV > GAN+DPS, with a particularly marked gap in the case of the generative method.

10. Discussion and Critical Comparison Between Methods

10.1 Strengths and Limitations of Each Method

Total p-Variation (Variational Method)

- **Strengths**: it requires no training data, is interpretable (a single regularization hyperparameter and an exponent p to choose), and is robust and generalizes by construction to any image, with no risk of overfitting to a specific dataset;
- **Limitations**: performance remains lower than a well-trained supervised model (about 6 dB less PSNR than the U-Net), choosing and p requires a dedicated grid search for each noise/scale condition, and the method tends to lose fine texture detail, especially for higher scale factors.

Attention U-Net (End-to-End Method)

- **Strengths**: clearly superior performance in every tested configuration, thanks to the network's ability to learn the inverse map directly from data, leveraging residual learning and the attention mechanism to balance denoising with detail preservation;
- **Limitations**: it requires task-specific supervised training for each combination of scale and noise (4 independent models), with a consequent need for labeled data (HR/LR pairs) and lower generalizability to unseen degradation conditions compared to the variational method.

GAN + DPS (Generative Method)

- **Strengths**: it requires no training conditioned on the specific super-resolution task (the GAN is trained only once, unconditionally, and reused for any degradation operator K at inference time via optimization of the latent vector alone);
- **Limitations**: the obtained performance is clearly inferior to both the U-Net and the TpV method, since reconstruction quality is bounded by the expressiveness (and possible lack of support coverage) of the pre-trained generator; the latent-optimization process is also computationally expensive (1000 steps per image) and sensitive to the choice of step-size and noise-schedule hyperparameters.

10.2 Summary of the Comparison

The experimental comparison highlights a clear hierarchy among the three methods in terms of quantitative performance: the **U-Net** (End-to-End) turns out to be the best-performing method overall, the **TpV** method (variational) represents a solid trade-off with no supervised training, while the **GAN+DPS** method (generative), although conceptually interesting for its task-agnostic nature, shows the typical limitations of inversion approaches based on unconditional generative priors when the generator is not sufficiently expressive or has not been specifically trained for the domain and resolution of the problem at hand.

11. Conclusions

- **U-Net (End-to-End)** is the best-performing method overall: PSNR ranging between 33 dB and nearly 40 dB, with SSIM consistently above 0.91, clearly outperforming the other two methods in every tested configuration;
- **TpV (Variational)** is confirmed as a solid runner-up: it stays within about 6 dB of the U-Net despite using no training data at all, a remarkable result for a purely model-based method;
- **GAN + DPS (Generative)** is the weakest method (about 20 dB PSNR), producing overly smooth reconstructions that lose fine texture detail: the identified bottleneck is the expressiveness of the frozen generator, not the severity of the applied degradation.

Overall, the project made it possible to systematically and reproducibly compare three profoundly different methodological paradigms for the same inverse problem, highlighting how the performance advantage of supervised methods (U-Net) comes together with lower flexibility compared to model-based methods (TpV), and how the use of a pre-trained generative prior (GAN+DPS), although conceptually elegant, requires a sufficiently powerful and expressive generator to compete with the other two families of approaches on this specific task.

11.1 Possible Future Developments

- Extending the comparison to the fourth method required by the assignment (Plug-and-Play in Half Quadratic Splitting), to complete the overview of all four methodological families;
- Training a more expressive GAN generator (at higher resolution or with greater capacity), or replacing it with a native diffusion model, for a fairer comparison with the U-Net;
- Evaluation on a larger number of test images (the entire test set, rather than just the first batch) for a more robust estimate of the average metrics;
- A more extensive sensitivity analysis on the hyperparameters of each method (number of DPS steps, noise schedule, additional and p values for TpV).

12. Bibliography and References

Below is a list of the main methodological references underlying the methods implemented in this project. Note that the complete list of cited sources, with precise bibliographic references, is to be completed in the final version of the report, as also indicated in the presentation associated with the project.

- Chambolle, A., Pock, T. — *A First-Order Primal-Dual Algorithm for Convex Problems with Applications to Imaging*, for the algorithm underlying the variational TpV method;
- Ronneberger, O., Fischer, P., Brox, T. — *U-Net: Convolutional Networks for Biomedical Image Segmentation*, for the base encoder-decoder architecture;
- Oktay, O. et al. — *Attention U-Net: Learning Where to Look for the Pancreas*, for the Attention Gate mechanism used in the decoder;
- Goodfellow, I. et al. — *Generative Adversarial Networks*, for the base adversarial generative framework;
- Chung, H. et al. — *Diffusion Posterior Sampling for General Noisy Inverse Problems*, for the DPS framework adapted in this project to GAN-based inversion via latent optimization;
- McCollough, C. — *Mayo Clinic Low-Dose CT Grand Challenge Dataset*, for the CT image dataset used.